

LAPORAN PRAKTIKUM MONGODB PIPELINE AGREGASI



Dosen Pengampu : M. Harry Khomas Saputra S.T, M.TI.
Nama : Fatih Akmal Alqathani
NIM : 225443009
Kelas : 1AEC1



**TEKNOLOGI REKAYASA INFORMATIKA INDUSTRI
POLITEKNIK MANUFAKTUR BANDUNG
TAHUN AJARAN 2025/2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM
MONGODB BASIC OPERATIONS

Judul laporan ini telah disetujui dan disahkan
di Politeknik Manufaktur Bandung
Pada Tanggal: 10 Maret 2026

NAMA	: FATIH AKMAL ALQATHANI
NOMOR INDUK MAHASISWA	: 225443009
KELAS	: 1AEC1
KONSENTRASI KEAHLIAN	: TEKNOLOGI REKAYASA INFORMATIKA INDUTRI

Mengetahui /menyetujui:

Kepala Prodi
Teknologi Rekayasa Informatika

Dosen Mata Kuliah
Praktikum Basis Data

Siti Aminah

M. Harry Khomas Saputra S.T, M,TL.

PLP Mata Kuliah
Praktikum Basis Data

Anis Rosiani

KATA PENGANTAR

Puji syukur kehadiran Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya, sehingga penulis dapat menyelesaikan laporan praktikum basis data ini tepat pada waktunya. Laporan ini disusun untuk memenuhi tugas mata kuliah dengan fokus materi pada implementasi basis data NoSQL menggunakan MongoDB, yang mencakup teknik otomatisasi data (data ingestion) dengan Python, pemrosesan analisis pabrik menggunakan fitur Aggregation Framework, hingga pemahaman struktur desain JSON Schema.

Penulis mengucapkan terima kasih kepada semua pihak yang telah membantu dalam pelaksanaan praktikum dan penyusunan laporan ini, khususnya kepada:

1. Bapak M. Harry Khomas Saputra selaku dosen pengampu yang sudah membimbing dalam pembelajaran mata kuliah Praktikum Basis Data.
2. Ibu Anis Rosiani selaku PLP Mata Kuliah Praktikum Basis Data yang telah mendampingi dan membantu dalam pelaksanaan praktikum.
3. Ibu Siti Aminah selaku Kepala Program Studi Teknologi Rekayasa Informatika yang telah memberikan fasilitas dan dukungan dalam kegiatan praktikum.
4. Semua pihak terutama rekan-rekan yang tidak dapat disebutkan satu per satu yang telah membantu dalam penyelesaian laporan ini.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, kritik dan saran yang membangun sangat penulis harapkan untuk perbaikan di masa mendatang. Semoga laporan ini dapat bermanfaat bagi pembaca dan menjadi referensi yang berguna dalam mempelajari MongoDB dan basis data NoSQL.

Bandung, 04 Mei 2026

Penulis

DAFTAR ISI

KATA PENGANTAR	2
DAFTAR ISI	3
BAB I PENDAHULUAN	5
1.1 Latar Belakang	5
1.2 Tujuan Praktikum	6
BAB II LATIHAN TERBIMBING.....	7
2.1 Desain dan Persiapan Data	7
2.1.1 Desain Data.....	7
2.1.2 Persiapan Data	9
2.2 Menghitung OEE perbulan	10
2.3 Menemukan Mesin dengan OEE di Bawah 80%.....	12
2.4 Distribusi Jumlah Produksi per Shift (Bucket)	14
2.5 Latihan Tambahan	16
2.5.1 Tren Bulanan (OEE semua mesin)	16
2.5.2 Mesin Terburuk (Top 3 terendah di Juni 2026).....	17
2.5.3 Kinerja Harian	18
2.5.4 Simulasi Perbaikan (Filter Quality < 95%)	19
BAB III TUGAS MANDIRI	20
3.1 Membuat Database Koleksi	20
3.2 Pembuatan Data Dummy Menggunakan Python	20
3.3 Memastikan Data	22
3.4 Pipeline 1: Persentase Cacat per Batch	22
3.5 Pipeline 2: Gabung dengan Target dan Tentukan Status Batch.....	25
3.6 Pipeline 3: Jumlah Batch NOT OK per Mesin per Minggu	27

3.7	Pipeline 4: Top 3 Mesin dengan Jumlah NOT OK Terbanyak (bulan tertentu)....	28
3.8	Pipeline 5: Ringkasan Bulanan dengan \$facet.....	29
3.9	Analisis Keseluruhan	30
3.9	Kesimpulan	30
BAB IV TUGAS MANDIRI		31
4.1	Persiapan Database dan Data Dummy	31
4.2	Rata-rata nilai per sensor_id pada 7 hari terakhir	33
4.3	Sensor dengan nilai di atas 90 minimal 3 kali	34
4.4	Menambahkan field jam dan menghitung rata-rata per jam per sensor.....	35
4.5	Membuat koleksi baru lalu JOIN (\$lookup)	36
BAB V KESIMPULAN.....		37

BAB I

PENDAHULUAN

1.1 Latar Belakang

Penguasaan database merupakan kompetensi fundamental yang harus dimiliki oleh setiap pengembang aplikasi di era digital saat ini. Berbagai macam aplikasi kini menggunakan system database untuk mengelola data. Kualitas dari sebuah aplikasi bisa dinilai dari kemampuan database itu sendiri, bagaimana performa nya, skalabilitas nya, dan fleksibilitas nya.

Ada dua metode utama dalam penyusunan database, yaitu database relasional SQL dan database NoSQL. Database relasional menggunakan tabel dengan skema tetap dan hubungan antar tabel, cocok untuk aplikasi dengan struktur data yang konsisten. Sementara itu, database NoSQL seperti MongoDB menawarkan fleksibilitas lebih dengan skema dinamis, mampu menyimpan data tidak terstruktur, serta mendukung skalabilitas horizontal yang lebih baik. MongoDB khususnya menggunakan format dokumen JSON (BSON) yang intuitif dan mudah dipahami oleh pengembang.

Pemilihan MongoDB sebagai objek praktikum didasarkan pada beberapa pertimbangan. Pertama, MongoDB merupakan database NoSQL paling populer saat ini dengan komunitas besar dan dokumentasi lengkap. Kedua, penggunaan format dokumen JSON memudahkan mahasiswa dalam memahami struktur data karena mirip dengan objek pada bahasa pemrograman modern seperti JavaScript dan Python.

1.2 Tujuan Praktikum

Tujuan yang ingin dicapai dalam pelaksanaan praktikum ini adalah sebagai berikut:

- a. Mahasiswa terampil membuat dataset simulasi yang mencerminkan data inspeksi kualitas di pabrik.
- b. Mahasiswa mampu merancang pipeline agregasi multi-stage menggunakan \$group, \$lookup, \$match, \$project, \$sort, \$addFields, dan \$facet.
- c. Dapat mengoptimasi pipeline dengan menempatkan \$match di awal dan memanfaatkan indeks.
- d. Mampu menginterpretasikan hasil agregasi untuk pengambilan keputusan mutu.

BAB II

LATIHAN TERBIMBING

2.1 Desain dan Persiapan Data

2.1.1 Desain Data

1. Collection data_sensor

Nama Field	Tipe Data	Wajib (Required)	Keterangan
_id	ObjectId	Ya	<i>Primary Key</i> bawaan MongoDB untuk setiap dokumen. Sistem memvalidasi bahwa isinya harus berupa 24 karakter.
category	String	Ya	Digunakan untuk menyimpan kategori atau jenis dari sensor tersebut (seperti "suhu", "tekanan", "getaran", atau "kelembaban"). Berupa teks karakter standar.
nilai	Double	Ya	Menyimpan hasil pembacaan spesifik dari sensor. Tipe <i>Double</i> menunjukkan bahwa <i>field</i> ini menampung angka desimal secara presisi.
sensor_id	String	Ya	Menyimpan kode identitas dari masing-masing unit sensor (contohnya "S01"). Meskipun berupa ID, tipe datanya adalah teks (<i>String</i>).
timestamp	Date	Ya	Menyimpan waktu (<i>date-time</i>). <i>Field</i> ini mengunci waktu yang sangat krusial untuk melacak kapan tepatnya sensor mencatat nilai tersebut.

2. Collection sensor_info

Nama Field	Tipe Data	Required	Keterangan
_id	ObjectId	Ya	Primary Key bawaan MongoDB untuk setiap dokumen di koleksi ini. Divalidasi secara ketat harus berupa 24 karakter.
lokasi	String	Ya	Menyimpan informasi letak fisik atau area penempatan sensor tersebut di pabrik (misalnya "Gedung 1" atau "Ruang Mesin"). Berupa teks karakter standar.
nama_sensor	String	Ya	Menyimpan nama deskriptif dari sensor agar mudah dibaca dan dipahami oleh teknisi (misalnya "Suhu Mesin A" atau "Tekanan Pompa B").
sensor_id	String	Ya	Menyimpan kode identitas dari unit sensor (contohnya "S01"). Field ini krusial karena bertindak layaknya Foreign Key untuk mengaitkan (JOIN melalui \$lookup) dokumen ini dengan data yang ada di koleksi data_sensor.

2.1.2 Persiapan Data

Pada desain dan persiapan data, saya menggunakan python untuk membuat data dummy sesuai instruksi, sebagai berikut:

```
PS C:\Users\Wheal> mongosh
Current Mongosh Log ID: 69f823789ad46fbcdd7c2986
Connecting to: mongod://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.7.0
Using MongoDB: 8.2.5
Using Mongosh: 2.7.0
mongosh 2.8.3 is available for download: https://www.mongodb.com/try/download/shell

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
The server generated these startup warnings when booting
2026-05-03T21:43:02.175+07:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----
test> use studi_kasus_oe
switched to db studi_kasus_oe
studi_kasus_oe>
```

Program python:

```
from pymongo import MongoClient
from datetime import datetime, timedelta
import random

client = MongoClient('mongodb://localhost:27017')
db = client['studi_kasus_oe']
collection = db['produksi_harian']
collection.drop() # Mulai bersih

mesin_list = [f"M{i:02d}" for i in range(1, 11)]
start_date = datetime(2026, 4, 1)
docs = []

for i in range(1000):
    # Acak hari dalam 90 hari
    day_offset = random.randint(0, 90)
    tanggal = start_date + timedelta(days=day_offset)
    shift = random.randint(1, 3)
    mesin = random.choice(mesin_list)
    target = random.randint(200, 500)
    actual_ok = int(target * random.uniform(0.6, 1.0)) # Kadang di bawah target
    actual_reject = int(actual_ok * random.uniform(0, 0.15)) # reject sampai 15%
    durasi_tersedia = 480 # 8 jam
    durasi_operasi = int(durasi_tersedia * random.uniform(0.7, 1.0))

    doc = {
        "mesin": mesin,
        "tanggal": tanggal,
        "shift": shift,
        "target": target,
        "actual_ok": actual_ok,
        "actual_reject": actual_reject,
        "durasi_operasi_menit": durasi_operasi,
        "durasi_tersedia_menit": durasi_tersedia
    }
    docs.append(doc)

    if len(docs) >= 500:
        collection.insert_many(docs)
        docs.clear()

if docs:
    collection.insert_many(docs)

print("Data berhasil dibuat. Total:", collection.count_documents({}))
```

2.2 Menghitung OEE perbulan

Menggunakan \$group untuk total data, dan \$project untuk rumus kalkulasi matematika OEE.

Program:

```
db.produksi_harian.aggregate([
  { $group: {
    _id: {
      mesin: "$mesin",
      bulan: { $month: "$tanggal" },
      tahun: { $year: "$tanggal" }
    },
    total_target: { $sum: "$target" },
    total_ok: { $sum: "$actual_ok" },
    total_reject: { $sum: "$actual_reject" },
    total_op: { $sum: "$durasi_operasi_menit" },
    total_avail: { $sum: "$durasi_tersedia_menit" }
  }},
  { $project: {
    _id: 0,
    mesin: "$_id.mesin",
    bulan: "$_id.bulan",
    tahun: "$_id.tahun",
    total_ok: 1,
    total_target: 1,
    total_reject: 1,
    availability: { $divide: ["$total_op", "$total_avail"] },
    performance: { $divide: ["$total_ok", "$total_target"] },
    quality: { $divide: ["$total_ok", { $add: ["$total_ok", "$total_reject"] } ] },
    OEE: {
      $multiply: [
        { $divide: ["$total_ok", "$total_target"] },
        { $divide: ["$total_op", "$total_avail"] },
        { $divide: ["$total_ok", { $add: ["$total_ok", "$total_reject"] } ] }
      ]
    }
  }},
  { $sort: { OEE: 1 } }
])
```

Output:

```
[
  {
    total_target: 10689,
    total_ok: 8113,
    total_reject: 571,
    mesin: 'M09',
    bulan: 5,
    tahun: 2026,
    availability: 0.816796875,
    performance: 0.7590045841519318,
    quality: 0.9342468908337171,
    OEE: 0.5791887632720145
  },
  {
    total_target: 5206,
    total_ok: 3805,
    total_reject: 224,
    mesin: 'M06',
    bulan: 4,
    tahun: 2026,
    availability: 0.8494047619047619,
    performance: 0.7308874375720322,
    quality: 0.9444030776867709,
    OEE: 0.5863036291714206
  },
  {
    total_target: 13294,
    total_ok: 10121,
    total_reject: 862,
    mesin: 'M08',
    bulan: 6,
    tahun: 2026,
    availability: 0.8378048780487805,
    performance: 0.761320896645103,
    quality: 0.9215150687426023,
    OEE: 0.5877776610556976
  },
  Dst...
]
```

2.3 Menemukan Mesin dengan OEE di Bawah 80%

Menambahkan stage \$match setelah kalkulasi \$project.

Program:

```
db.produksi_harian.aggregate([
  { $group: {
    _id: {
      mesin: "$mesin",
      bulan: { $month: "$tanggal" },
      tahun: { $year: "$tanggal" }
    },
    total_target: { $sum: "$target" },
    total_ok: { $sum: "$actual_ok" },
    total_reject: { $sum: "$actual_reject" },
    total_op: { $sum: "$durasi_operasi_menit" },
    total_avail: { $sum: "$durasi_tersedia_menit" }
  }},
  { $project: {
    _id: 0,
    mesin: "$_id.mesin",
    bulan: "$_id.bulan",
    tahun: "$_id.tahun",
    OEE: {
      $multiply: [
        { $divide: ["$total_ok", "$total_target" ] },
        { $divide: ["$total_op", "$total_avail" ] },
        { $divide: ["$total_ok", { $add: ["$total_ok", "$total_reject" ] } ] }
      ]
    }
  }},
  { $match: { OEE: { $lt: 0.80 } } },
  { $sort: { OEE: 1 } }
])
```

Output:

```
[
  { mesin: 'M09', bulan: 5, tahun: 2026, OEE: 0.5791887632720145 },
  { mesin: 'M06', bulan: 4, tahun: 2026, OEE: 0.5863036291714206 },
  { mesin: 'M08', bulan: 6, tahun: 2026, OEE: 0.5877776610556976 },
  { mesin: 'M09', bulan: 6, tahun: 2026, OEE: 0.5878679171285299 },
  { mesin: 'M01', bulan: 4, tahun: 2026, OEE: 0.603845686567223 },
  { mesin: 'M06', bulan: 6, tahun: 2026, OEE: 0.6042872451101061 },
  { mesin: 'M03', bulan: 5, tahun: 2026, OEE: 0.6081113156523538 },
  { mesin: 'M03', bulan: 6, tahun: 2026, OEE: 0.6138596418069736 },
  { mesin: 'M04', bulan: 5, tahun: 2026, OEE: 0.6141811078637452 },
  { mesin: 'M05', bulan: 4, tahun: 2026, OEE: 0.6167606757454258 },
  { mesin: 'M08', bulan: 5, tahun: 2026, OEE: 0.6171358156682882 },
  { mesin: 'M02', bulan: 4, tahun: 2026, OEE: 0.6215375277432397 },
  { mesin: 'M06', bulan: 5, tahun: 2026, OEE: 0.6250468826564841 },
  { mesin: 'M07', bulan: 5, tahun: 2026, OEE: 0.6257398047128512 },
  { mesin: 'M01', bulan: 5, tahun: 2026, OEE: 0.6261097092971368 },
  { mesin: 'M08', bulan: 4, tahun: 2026, OEE: 0.6274830553465168 },
  { mesin: 'M10', bulan: 4, tahun: 2026, OEE: 0.6288290340755407 },
  { mesin: 'M03', bulan: 4, tahun: 2026, OEE: 0.6323686244856639 },
  { mesin: 'M10', bulan: 5, tahun: 2026, OEE: 0.6339766528704425 },
  { mesin: 'M02', bulan: 6, tahun: 2026, OEE: 0.6371598527223353 }
]
```

2.4 Distribusi Jumlah Produksi per Shift (Bucket)

Menggunakan kombinasi \$facet untuk multi-pipeline per shift dan \$bucket untuk rentang nilai.

Program:

```
db.produksi_harian.aggregate([
  { $facet: {
    "shift1": [
      { $match: { shift: 1 } },
      { $bucket: {
        groupBy: "$actual_ok",
        boundaries: [0, 200, 300, 400, 600],
        default: "600+",
        output: { count: { $sum: 1 } }
      } }
    ],
    "shift2": [
      { $match: { shift: 2 } },
      { $bucket: {
        groupBy: "$actual_ok",
        boundaries: [0, 200, 300, 400, 600],
        default: "600+",
        output: { count: { $sum: 1 } }
      } }
    ],
    "shift3": [
      { $match: { shift: 3 } },
      { $bucket: {
        groupBy: "$actual_ok",
        boundaries: [0, 200, 300, 400, 600],
        default: "600+",
        output: { count: { $sum: 1 } }
      } }
    ]
  } }
])
```

Output:

```
[
  {
    shift1: [
      { _id: 0, count: 60 },
      { _id: 200, count: 160 },
      { _id: 300, count: 96 },
      { _id: 400, count: 20 }
    ],
    shift2: [
      { _id: 0, count: 64 },
      { _id: 200, count: 146 },
      { _id: 300, count: 95 },
      { _id: 400, count: 29 }
    ],
    shift3: [
      { _id: 0, count: 60 },
      { _id: 200, count: 137 },
      { _id: 300, count: 104 },
      { _id: 400, count: 29 }
    ]
  }
]
```


2.5 Latihan Tambahan

2.5.1 Tren Bulanan (OEE semua mesin)

Program:

```
db.produksi_harian.aggregate([
  { $group: {
    _id: { bulan: { $month: "$tanggal" }, tahun: { $year: "$tanggal" } },
    total_target: { $sum: "$target" },
    total_ok: { $sum: "$actual_ok" },
    total_reject: { $sum: "$actual_reject" },
    total_op: { $sum: "$durasi_operasi_menit" },
    total_avail: { $sum: "$durasi_tersedia_menit" }
  }},
  { $project: {
    _id: 0,
    bulan: "$_id.bulan",
    tahun: "$_id.tahun",
    OEE: {
      $multiply: [
        { $divide: ["$total_ok", "$total_target" ] },
        { $divide: ["$total_op", "$total_avail" ] },
        { $divide: ["$total_ok", { $add: ["$total_ok", "$total_reject" ] } ] }
      ]
    }
  }},
  { $sort: { tahun: 1, bulan: 1 } }
])
```

Output :

```
[
  { bulan: 4, tahun: 2026, OEE: 0.6307361864099431 },
  { bulan: 5, tahun: 2026, OEE: 0.6256010265940415 },
  { bulan: 6, tahun: 2026, OEE: 0.6282824482348016 }
]
```

2.5.2 Mesin Terburuk (Top 3 terendah di Juni 2026)

Program:

```
db.produksi_harian.aggregate([
  { $match: {
    tanggal: { $gte: ISODate("2026-06-01"), $lt: ISODate("2026-07-01") }
  }},
  { $group: {
    _id: "$mesin",
    total_target: { $sum: "$target" },
    total_ok: { $sum: "$actual_ok" },
    total_reject: { $sum: "$actual_reject" },
    total_op: { $sum: "$durasi_operasi_menit" },
    total_avail: { $sum: "$durasi_tersedia_menit" }
  }},
  { $project: {
    _id: 0,
    mesin: "$_id",
    OEE: {
      $multiply: [
        { $divide: [ "$total_ok", "$total_target" ] },
        { $divide: [ "$total_op", "$total_avail" ] },
        { $divide: [ "$total_ok", { $add: [ "$total_ok", "$total_reject" ] } ] }
      ]
    }
  }},
  { $sort: { OEE: 1 } },
  { $limit: 3 }
])
```

Output:

```
[
  { mesin: 'M08', OEE: 0.5877776610556976 },
  { mesin: 'M09', OEE: 0.5878679171285299 },
  { mesin: 'M06', OEE: 0.6042872451101061 }
]
```

2.5.3 Kinerja Harian

Program:

```
db.produksi_harian.aggregate([
  { $group: {
    _id: {
      mesin: "$mesin",
      tanggal_produksi: { $dateToString: { format: "%Y-%m-%d", date: "$tanggal" } }
    },
    total_target: { $sum: "$target" },
    total_ok: { $sum: "$actual_ok" },
    total_reject: { $sum: "$actual_reject" },
    total_op: { $sum: "$durasi_operasi_menit" },
    total_avail: { $sum: "$durasi_tersedia_menit" }
  }},
  { $project: {
    _id: 0,
    mesin: "$_id.mesin",
    tanggal: "$_id.tanggal_produksi",
    OEE: {
      $multiply: [
        { $divide: ["$total_ok", "$total_target"] },
        { $divide: ["$total_op", "$total_avail"] },
        { $divide: ["$total_ok", { $add: ["$total_ok", "$total_reject"] } ] }
      ]
    }
  }},
  { $sort: { tanggal: 1, mesin: 1 } }
])
```

Output:

```
[
  { mesin: 'M01', tanggal: '2026-04-01', OEE: 0.47486251588533246 },
  { mesin: 'M02', tanggal: '2026-04-01', OEE: 0.6362347826086956 },
  { mesin: 'M03', tanggal: '2026-04-01', OEE: 0.7067289071495865 },
  { mesin: 'M04', tanggal: '2026-04-01', OEE: 0.5436552164651 },
  { mesin: 'M05', tanggal: '2026-04-01', OEE: 0.8000221761792224 },
  { mesin: 'M06', tanggal: '2026-04-01', OEE: 0.6594973465945034 },
  { mesin: 'M07', tanggal: '2026-04-01', OEE: 0.7710682031671832 },
  { mesin: 'M08', tanggal: '2026-04-01', OEE: 0.569891965202308 },
  { mesin: 'M09', tanggal: '2026-04-01', OEE: 0.5162211789272 },
  { mesin: 'M10', tanggal: '2026-04-01', OEE: 0.6023787730040013 },
  { mesin: 'M01', tanggal: '2026-04-02', OEE: 0.6871682769657345 },
  { mesin: 'M02', tanggal: '2026-04-02', OEE: 0.6739380461922597 },
  { mesin: 'M03', tanggal: '2026-04-02', OEE: 0.7242231269791558 },
  { mesin: 'M04', tanggal: '2026-04-02', OEE: 0.753853713237346 },
  { mesin: 'M07', tanggal: '2026-04-02', OEE: 0.5479585804356026 },
  { mesin: 'M08', tanggal: '2026-04-02', OEE: 0.687876181602392 },
  { mesin: 'M09', tanggal: '2026-04-02', OEE: 0.7600473701794579 },
  { mesin: 'M10', tanggal: '2026-04-02', OEE: 0.8609173828280418 },
  { mesin: 'M01', tanggal: '2026-04-03', OEE: 0.6470223370518738 },
  { mesin: 'M02', tanggal: '2026-04-03', OEE: 0.5997471970437513 }
]
```

2.5.4 Simulasi Perbaikan (Filter Quality < 95%)

Program:

```
db.produksi_harian.aggregate([
  { $group: {
    _id: "$mesin",
    total_ok: { $sum: "$actual_ok" },
    total_reject: { $sum: "$actual_reject" }
  }},
  { $project: {
    _id: 0,
    mesin: "$_id",
    quality: { $divide: [ "$total_ok", { $add: [ "$total_ok", "$total_reject" ] } ] }
  }},
  { $match: { quality: { $lt: 0.95 } } },
  { $sort: { quality: 1 } }
])
```

Output:

```
[
  { mesin: 'M08', quality: 0.9259182688994556 },
  { mesin: 'M02', quality: 0.9261159248021521 },
  { mesin: 'M04', quality: 0.9280969128565846 },
  { mesin: 'M10', quality: 0.9287856299033714 },
  { mesin: 'M01', quality: 0.9303080512629763 },
  { mesin: 'M03', quality: 0.9316377483205805 },
  { mesin: 'M07', quality: 0.9322201274800251 },
  { mesin: 'M05', quality: 0.9328201323192976 },
  { mesin: 'M06', quality: 0.9375276046523041 },
  { mesin: 'M09', quality: 0.9379864979525584 }
]
```

BAB III

TUGAS MANDIRI

3.1 Membuat Database Koleksi

```
PS C:\Users\Akmal> mongosh
Current Mongosh Log ID: 69f8419a28f0ef4cd27c2906
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.7.0
Using MongoDB:      8.2.5
Using Mongosh:       2.7.0
mongosh 2.8.3 is available for download: https://www.mongodb.com/try/download/shell

For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/

-----
  The server generated these startup warnings when booting
  2026-05-03T21:43:02.175+07:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----

test> use tugas5_225443009
switched to db tugas5_225443009
tugas5_225443009> |
```

3.2 Pembuatan Data Dummy Menggunakan Python

Program Python:

```
from pymongo import MongoClient
from datetime import datetime, timedelta
import random

# Ganti dengan NIM Anda
NIM = "225443009"
client = MongoClient('mongodb://localhost:27017')
db = client[f'tugas5_225443009']
db.inspeksi.drop()
db.target_kualitas.drop()

mesin_list = [f"M{i:02d}" for i in range(1, 11)]
batch_ids = [f"B{str(i).zfill(4)}" for i in range(1, 2001)]
inspektor_list = ["Andi", "Budi", "Citra", "Dian", "Eka"]
jenis_cacat_options = ["gores", "retak", "bengkok", "warna", "dimensi", "pori"]

# Generate data inspeksi (1500 dokumen)
docs = []
start_date = datetime(2026, 4, 1, 6, 0)
for i in range(1500):
    batch = random.choice(batch_ids)
    mesin = random.choice(mesin_list)
    tanggal = start_date + timedelta(minutes=random.randint(0, 60*24*90)) # 3 bulan
    shift = 1 if tanggal.hour < 14 else (2 if tanggal.hour < 22 else 3)
    jumlah = random.randint(100, 500)
    reject = int(jumlah * random.uniform(0, 0.10)) # 0-10% cacat
    jenis = random.sample(jenis_cacat_options, k=random.randint(1, 3))
    doc = {
        "batch_id": batch,
        "mesin": mesin,
        "tanggal": tanggal,
        "shift": shift,
        "inspektor": random.choice(inspektor_list),
        "jumlah_diperiksa": jumlah,
```

```

        "cacat_ditemukan": reject,
        "jenis_cacat": jenis
    }
    docs.append(doc)
    if len(docs) >= 500:
        db.inspeksi.insert_many(docs)
        docs.clear()
    if docs:
        db.inspeksi.insert_many(docs)

print("Data inspeksi selesai. Total:", db.inspeksi.count_documents({}))

# Generate target kualitas (setiap batch punya target cacat maksimal 5% dari rata-rata ukuran batch)
targets = []
for i in range(1, 2001):
    batch = f"B{str(i).zfill(4)}"
    # target cacat acak 1-15
    target_cacat = random.randint(1, 15)
    targets.append({"batch_id": batch, "target_maks_cacat": target_cacat})
    if len(targets) >= 500:
        db.target_kualitas.insert_many(targets)
        targets.clear()
    if targets:
        db.target_kualitas.insert_many(targets)

print("Data target kualitas selesai. Total:", db.target_kualitas.count_documents({}))

```

Output:

```

PS C:\Users\Akmal\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Akmal\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/Akmal/Downloads/from_pymongo_import_MongoClient.py"
Data inspeksi selesai. Total: 1500
Data target kualitas selesai. Total: 2000

```

3.3 Memastikan Data

```
tugas5_225443009> db.inspeksi.countDocuments() // harus >= 1000
1500
tugas5_225443009> db.target_kualitas.countDocuments() // harus >= 50
2000
tugas5_225443009> db.inspeksi.findOne()
{
  _id: ObjectId('69f8163d95a317b797498eac'),
  batch_id: 'B1108',
  mesin: 'M05',
  tanggal: ISODate('2026-06-11T19:01:00.000Z'),
  shift: 2,
  inspektor: 'Citra',
  jumlah_diperiksa: 375,
  cacat_ditemukan: 11,
  jenis_cacat: [ 'retak', 'gores', 'dimensi' ]
}
tugas5_225443009> db.target_kualitas.findOne()
{
  _id: ObjectId('69f8163d95a317b797499488'),
  batch_id: 'B0001',
  target_maks_cacat: 6
}
```

```
tugas5_225443009> use tugas5_225443009
|
| db.inspeksi.createIndex({ batch_id: 1 })
| db.inspeksi.createIndex({ mesin: 1, tanggal: 1 })
| db.target_kualitas.createIndex({ batch_id: 1 })
already on db tugas5_225443009
```

3.4 Pipeline 1: Persentase Cacat per Batch

Program:

```
db.inspeksi.aggregate([
|   { $addFields: {
|     persen_cacat: { $multiply: [ { $divide: ["$cacat_ditemukan", "$jumlah_diperiksa"] }, 100 ] }
|   } },
|   { $sort: { persen_cacat: -1 } },
|   // Masukkan $limit ke dalam sini sebagai stage terakhir
|   { $limit: 10 }
| ])
```

Output:

```
[
  {
    _id: ObjectId('69f8163d95a317b797498f50'),
    batch_id: 'B1703',
    mesin: 'M10',
    tanggal: ISODate('2026-04-06T15:42:00.000Z'),
    shift: 2,
    inspektor: 'Citra',
    jumlah_diperiksa: 291,
    cacat_ditemukan: 29,
    jenis_cacat: [ 'retak' ],
    persen_cacat: 9.965635738831615
  },
  {
    _id: ObjectId('69f8163d95a317b79749907a'),
    batch_id: 'B0108',
    mesin: 'M08',
    tanggal: ISODate('2026-05-03T20:47:00.000Z'),
    shift: 2,
    inspektor: 'Eka',
    jumlah_diperiksa: 422,
    cacat_ditemukan: 42,
    jenis_cacat: [ 'retak' ],
    persen_cacat: 9.95260663507109
  },
  {
    _id: ObjectId('69f8163d95a317b797498f6a'),
    batch_id: 'B1069',
    mesin: 'M07',
    tanggal: ISODate('2026-06-29T18:35:00.000Z'),
    shift: 2,
    inspektor: 'Dian',
    jumlah_diperiksa: 342,
    cacat_ditemukan: 34,
    jenis_cacat: [ 'retak', 'bengkok' ],
    persen_cacat: 9.941520467836257
  },
  {
    _id: ObjectId('69f8163d95a317b79749933c'),
    batch_id: 'B1562',
    mesin: 'M06',
    tanggal: ISODate('2026-04-12T02:44:00.000Z'),
    shift: 1,
    inspektor: 'Dian',
    jumlah_diperiksa: 292,
    cacat_ditemukan: 29,
    jenis_cacat: [ 'gores', 'pori', 'dimensi' ],
    persen_cacat: 9.931506849315069
  },
  {
    _id: ObjectId('69f8163d95a317b7974990a5'),
    batch_id: 'B0861',
    mesin: 'M07',
    tanggal: ISODate('2026-05-07T02:48:00.000Z'),
    shift: 1,
    inspektor: 'Budi',
    jumlah_diperiksa: 395,
    cacat_ditemukan: 39,
    jenis_cacat: [ 'bengkok', 'gores', 'warna' ],
    persen_cacat: 9.873417721518987
  },
  {
    _id: ObjectId('69f8163d95a317b7974992ca'),
    batch_id: 'B1806',
    mesin: 'M01',
    tanggal: ISODate('2026-04-07T11:55:00.000Z'),
    shift: 1,
    inspektor: 'Andi',
    jumlah_diperiksa: 385,
    cacat_ditemukan: 38,
    jenis_cacat: [ 'warna', 'retak' ],
    persen_cacat: 9.87012987012987
  },
  {
    _id: ObjectId('69f8163d95a317b79749942d'),
    batch_id: 'B0354',
    mesin: 'M10',
    tanggal: ISODate('2026-05-01T21:15:00.000Z'),
    shift: 2,
    inspektor: 'Dian',
    jumlah_diperiksa: 304,
    cacat_ditemukan: 30,
    jenis_cacat: [ 'warna', 'bengkok' ],
    persen_cacat: 9.868421052631579
  }
]
```



```
},
{
  _id: ObjectId('69f8163d95a317b7974992fc'),
  batch_id: 'B1672',
  mesin: 'M10',
  tanggal: ISODate('2026-04-28T09:23:00.000Z'),
  shift: 1,
  inspektor: 'Budi',
  jumlah_diperiksa: 315,
  cacat_ditemukan: 31,
  jenis_cacat: [ 'retak', 'warna' ],
  persen_cacat: 9.841269841269842
},
{
  _id: ObjectId('69f8163d95a317b7974991b5'),
  batch_id: 'B0537',
  mesin: 'M02',
  tanggal: ISODate('2026-05-07T18:01:00.000Z'),
  shift: 2,
  inspektor: 'Budi',
  jumlah_diperiksa: 244,
  cacat_ditemukan: 24,
  jenis_cacat: [ 'bengkok', 'gores', 'dimensi' ],
  persen_cacat: 9.836065573770492
},
{
  _id: ObjectId('69f8163d95a317b797499369'),
  batch_id: 'B0044',
  mesin: 'M04',
  tanggal: ISODate('2026-05-08T05:32:00.000Z'),
  shift: 1,
  inspektor: 'Budi',
  jumlah_diperiksa: 397,
  cacat_ditemukan: 39,
  jenis_cacat: [ 'warna' ],
  persen_cacat: 9.82367758186398
}
]
```

3.5 Pipeline 2: Gabung dengan Target dan Tentukan Status Batch

Program :

```
db.inspeksi.aggregate([
  { $lookup: {
    from: "target_kualitas",
    localField: "batch_id",
    foreignField: "batch_id",
    as: "target_info"
  }},
  { $unwind: "$target_info" },
  { $addFields: {
    persen_cacat: { $multiply: [ { $divide: ["$cacat_ditemukan", "$jumlah_diperiksa" ] }, 100 ] },
    target_maks: "$target_info.target_maks_cacat"
  }},
  { $addFields: {
    status: { $cond: { if: { $gt: ["$persen_cacat", "$target_maks" ] }, then: "NOT OK", else: "OK" } }
  }},
  { $project: {
    _id: 0,
    batch_id: 1,
    mesin: 1,
    persen_cacat: 1,
    target_maks: 1,
    status: 1,
    selisih: { $subtract: ["$persen_cacat", "$target_maks" ] }
  }},
  { $sort: { selisih: -1 } },
  // Masukkan $limit ke dalam sini sebagai stage terakhir
  { $limit: 10 }
])
```

Output:

```
[
  {
    batch_id: 'B0044',
    mesin: 'M04',
    persen_cacat: 9.82367758186398,
    target_maks: 1,
    status: 'NOT OK',
    selisih: 8.82367758186398
  },
  {
    batch_id: 'B1558',
    mesin: 'M09',
    persen_cacat: 9.565217391304348,
    target_maks: 1,
    status: 'NOT OK',
    selisih: 8.565217391304348
  },
  {
    batch_id: 'B1712',
    mesin: 'M10',
    persen_cacat: 9.433962264150944,
    target_maks: 1,
    status: 'NOT OK',
    selisih: 8.433962264150944
  },
  {
    batch_id: 'B0165',
    mesin: 'M04',
    persen_cacat: 9.345794392523365,
    target_maks: 1,
    status: 'NOT OK',
    selisih: 8.345794392523365
  },
  {
    batch_id: 'B0522',
    mesin: 'M08',
    persen_cacat: 9.13978494623656,
    target_maks: 1,
    status: 'NOT OK',
    selisih: 8.13978494623656
  },
  {
    batch_id: 'B1171',
    mesin: 'M07',
    persen_cacat: 9.112709832134293,
    target_maks: 1,
    status: 'NOT OK',
    selisih: 8.112709832134293
  }
]
```

```
},
{
  batch_id: 'B0657',
  mesin: 'M10',
  persen_cacat: 9.003215434083602,
  target_maks: 1,
  status: 'NOT OK',
  selisih: 8.003215434083602
},
{
  batch_id: 'B1517',
  mesin: 'M02',
  persen_cacat: 9.79020979020979,
  target_maks: 2,
  status: 'NOT OK',
  selisih: 7.79020979020979
},
{
  batch_id: 'B0041',
  mesin: 'M02',
  persen_cacat: 8.633093525179856,
  target_maks: 1,
  status: 'NOT OK',
  selisih: 7.633093525179856
},
{
  batch_id: 'B0247',
  mesin: 'M03',
  persen_cacat: 8.614232209737828,
  target_maks: 1,
  status: 'NOT OK',
  selisih: 7.614232209737828
}
]
tugas5_
```

3.6 Pipeline 3: Jumlah Batch NOT OK per Mesin per Minggu

Program:

```
db.inspeksi.aggregate([
  { $lookup: {
    from: "target_kualitas",
    localField: "batch_id",
    foreignField: "batch_id",
    as: "target_info"
  }},
  { $unwind: "$target_info" },
  { $addFields: {
    persen_cacat: { $multiply: [ { $divide: [ "$cacat_ditemukan", "$jumlah_diperiksa" ] }, 100 ] }
  }},
  { $addFields: {
    status: { $cond: { if: { $gt: [ "$persen_cacat", "$target_info.target_maks_cacat" ] }, then: "NOT OK", else: "OK" } }
  }},
  { $match: { status: "NOT OK" } },
  { $group: {
    _id: {
      mesin: "$mesin",
      minggu: { $week: "$tanggal" },
      tahun: { $year: "$tanggal" }
    },
    jumlah_NOT_OK: { $sum: 1 }
  }},
  { $sort: { "_id.tahun": 1, "_id.minggu": 1, "_id.mesin": 1 } }
])
```

Output:

```
[
  { _id: { mesin: 'M01', minggu: 13, tahun: 2026 }, jumlah_NOT_OK: 4 },
  { _id: { mesin: 'M02', minggu: 13, tahun: 2026 }, jumlah_NOT_OK: 2 },
  { _id: { mesin: 'M04', minggu: 13, tahun: 2026 }, jumlah_NOT_OK: 4 },
  { _id: { mesin: 'M05', minggu: 13, tahun: 2026 }, jumlah_NOT_OK: 2 },
  { _id: { mesin: 'M06', minggu: 13, tahun: 2026 }, jumlah_NOT_OK: 3 },
  { _id: { mesin: 'M07', minggu: 13, tahun: 2026 }, jumlah_NOT_OK: 5 },
  { _id: { mesin: 'M08', minggu: 13, tahun: 2026 }, jumlah_NOT_OK: 2 },
  { _id: { mesin: 'M09', minggu: 13, tahun: 2026 }, jumlah_NOT_OK: 1 },
  { _id: { mesin: 'M10', minggu: 13, tahun: 2026 }, jumlah_NOT_OK: 2 },
  { _id: { mesin: 'M01', minggu: 14, tahun: 2026 }, jumlah_NOT_OK: 4 },
  { _id: { mesin: 'M02', minggu: 14, tahun: 2026 }, jumlah_NOT_OK: 1 },
  { _id: { mesin: 'M03', minggu: 14, tahun: 2026 }, jumlah_NOT_OK: 5 },
  { _id: { mesin: 'M04', minggu: 14, tahun: 2026 }, jumlah_NOT_OK: 1 },
  { _id: { mesin: 'M05', minggu: 14, tahun: 2026 }, jumlah_NOT_OK: 6 },
  { _id: { mesin: 'M06', minggu: 14, tahun: 2026 }, jumlah_NOT_OK: 4 },
  { _id: { mesin: 'M07', minggu: 14, tahun: 2026 }, jumlah_NOT_OK: 4 },
  { _id: { mesin: 'M08', minggu: 14, tahun: 2026 }, jumlah_NOT_OK: 1 },
  { _id: { mesin: 'M09', minggu: 14, tahun: 2026 }, jumlah_NOT_OK: 4 },
  { _id: { mesin: 'M10', minggu: 14, tahun: 2026 }, jumlah_NOT_OK: 4 },
  { _id: { mesin: 'M01', minggu: 15, tahun: 2026 }, jumlah_NOT_OK: 1 }
]
```

3.7 Pipeline 4: Top 3 Mesin dengan Jumlah NOT OK Terbanyak (bulan tertentu)

Program:

```
db.inspeksi.aggregate([
  { $match: {
    tanggal: { $gte: ISODate("2026-05-01"), $lt: ISODate("2026-06-01") }
  }},
  { $lookup: {
    from: "target_kualitas",
    localField: "batch_id",
    foreignField: "batch_id",
    as: "target_info"
  }},
  { $unwind: "$target_info" },
  { $addFields: {
    persen_cacat: { $multiply: [ { $divide: ["$cacat_ditemukan", "$jumlah_diperiksa"] }, 100 ] }
  }},
  { $addFields: {
    status: { $cond: { if: { $gt: ["$persen_cacat", "$target_info.target_maks_cacat"] }, then: "NOT OK", else: "OK" } }
  }},
  { $match: { status: "NOT OK" } },
  { $group: {
    _id: "$mesin",
    total_NOT_OK: { $sum: 1 }
  }},
  { $sort: { total_NOT_OK: -1 } },
  { $limit: 3 }
])
```

Output:

```
[
  { _id: 'M09', total_NOT_OK: 18 },
  { _id: 'M04', total_NOT_OK: 17 },
  { _id: 'M02', total_NOT_OK: 16 }
]
```

3.8 Pipeline 5: Ringkasan Bulanan dengan \$facet

Program:

```
db.inspeksi.aggregate([
  { $match: { tanggal: { $gte: ISODate("2026-06-01"), $lt: ISODate("2026-07-01") } } },
  { $lookup: {
    from: "target_kualitas",
    localField: "batch_id",
    foreignField: "batch_id",
    as: "target_info"
  }},
  { $unwind: "$target_info" },
  { $addFields: {
    persen_cacat: { $multiply: [ { $divide: ["$cacat_ditemukan", "$jumlah_diperiksa"] }, 100 ] },
    status: { $cond: { if: { $gt: [ { $multiply: [ { $divide: ["$cacat_ditemukan", "$jumlah_diperiksa"] }, 100 ] },
    "$target_info.target_maks_cacat" ] }, then: "NOT OK", else: "OK" } }
  }},
  { $facet: {
    "total_batch": [
      { $count: "count" }
    ],
    "rata_cacat": [
      { $group: { _id: null, avg_cacat: { $avg: "$persen_cacat" } } }
    ],
    "mesin_terburuk": [
      { $match: { status: "NOT OK" } },
      { $group: { _id: "$mesin", jumlah: { $sum: 1 } } },
      { $sort: { jumlah: -1 } },
      { $limit: 3 }
    ]
  }
  }
  })
```

Output:

```
[
  {
    total_batch: [ { count: 485 } ],
    rata_cacat: [ { _id: null, avg_cacat: 4.9652325856705515 } ],
    mesin_terburuk: [
      { _id: 'M02', jumlah: 20 },
      { _id: 'M01', jumlah: 19 },
      { _id: 'M10', jumlah: 18 }
    ]
  }
]
```

3.9 Analisis Keseluruhan

Status Kualitas (Pipeline 1 & 2), sebagian besar produksi berjalan dengan baik, namun terdapat beberapa batch yang statusnya menembus batas maksimal target cacat (NOT OK). Selisih persentase cacat pada batch-batch ini menjadi indikator utama adanya masalah pada lini produksi pada hari tersebut.

Distribusi Kerusakan Mesin (Pipeline 3), melalui pengelompokan per minggu (\$week), terlihat bahwa mesin yang menghasilkan batch "NOT OK" cenderung berulang pada mesin yang sama dari minggu ke minggu. Ini mengindikasikan masalah komponen yang bersifat kronis, bukan sekadar anomali acak.

Fokus Masalah Bulan Mei & Juni (Pipeline 4 & 5) Adalah ringkasan operasional pada periode evaluasi khusus (Mei dan Juni 2026) memperlihatkan bahwa mesin M02 konsisten menjadi penyumbang yang gagal terbanyak di lantai pabrik.

3.9 Kesimpulan

Dari segi eksekusi program, fitur Aggregation MongoDB terbukti sangat ampuh untuk mengolah data kompleks langsung di dalam database. Lewat lima pipeline yang dibuat, kita bisa dengan cepat menghitung persentase barang cacat, membandingkannya dengan target batas maksimal menggunakan \$lookup, hingga membuat ringkasan laporan bulanan sekaligus menggunakan fitur \$facet.

Dari segi studi kasus, hasil pengolahan data tadi berhasil menemukan sumber masalah utama di pabrik. Walau produksi secara umum terlihat normal, ternyata ada tren kerusakan mesin yang terus berulang setiap minggunya. Berdasarkan data bulan Mei dan Juni 2026, Mesin M02 terbukti paling sering menghasilkan produk cacat (status NOT OK). Kesimpulannya, pabrik harus segera memprioritaskan perbaikan (maintenance) pada Mesin M02 agar standar kualitas produksi kembali terjaga.

BAB IV

LATIHAN MANDIRI

4.1 Persiapan Database dan Data Dummy

```
PS C:\Users\Akmal> mongosh
Current Mongosh Log ID: 69f85559b812d6ee367c2906
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.7.0
Using MongoDB:      8.2.5
Using Mongosh:       2.7.0
mongosh 2.8.3 is available for download: https://www.mongodb.com/try/download/shell

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
The server generated these startup warnings when booting
  2026-05-03T21:43:02.175+07:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----

test> use latihan5
switched to db latihan5
latihan5> []
```

Program Python:

```
from pymongo import MongoClient
from datetime import datetime, timedelta
import random

#Koneksi database
client = MongoClient('mongodb://localhost:27017')
db = client['latihan5']
collection = db['data_sensor']

# Persiapan dummy data
sensor_ids = [f"S{str(i).zfill(2)}" for i in range(1, 11)] # Menghasilkan S01 sampai S10
categories = ["suhu", "tekanan", "kelembaban", "getaran"]

# Patokan waktu: 14 hari ke belakang dari waktu saat program dijalankan
end_date = datetime.now()
start_date = end_date - timedelta(days=14)

docs = []

# Generate 5000 dokumen
for i in range(5000):
    # Acak tanggal dan waktu (timestamp)
    random_seconds = random.randint(0, int((end_date - start_date).total_seconds()))
    timestamp = start_date + timedelta(seconds=random_seconds)

    sensor_id = random.choice(sensor_ids)
    category = random.choice(categories)

    # Acak nilai sensor antara 20.0 hingga 100.0 (dibulatkan 2 angka di belakang koma)
    nilai = round(random.uniform(20.0, 100.0), 2)
```



```
doc = {
    "sensor_id": sensor_id,
    "nilai": nilai,
    "timestamp": timestamp,
    "category": category
}
docs.append(doc)

# Masukkan ke database per 500 data agar cepat dan stabil
if len(docs) >= 500:
    collection.insert_many(docs)
    docs.clear()

# Masukkan sisa data jika masih ada di dalam list
if docs:
    collection.insert_many(docs)

print("Data sensor berhasil dibuat. Total:", collection.count_documents({}))
```

Output:

```
PS C:\Users\Akmal\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Akmal\AppData\Local\Python\pythoncore-3.14-64\python.exe "C:/Users/Akmal/Downloads/from pymongo import MongoClient latihan mandiri.py"
Data sensor berhasil dibuat. Total: 5000
```

4.2 Rata-rata nilai per sensor_id pada 7 hari terakhir

Program:

```
// Asumsi tanggal hari ini adalah patokan, kita kurangi 7 hari ke belakang
var tujuhHariLalu = new Date();
tujuhHariLalu.setDate(tujuhHariLalu.getDate() - 7);

db.data_sensor.aggregate([
  // Stage 1: Filter data 7 hari terakhir
  { $match: {
    timestamp: { $gte: tujuhHariLalu }
  }},

  // Stage 2: Kelompokkan per sensor dan hitung rata-rata
  { $group: {
    _id: "$sensor_id",
    rata_rata_nilai: { $avg: "$nilai" }
  }},

  // Stage 3: Urutkan dari rata-rata tertinggi (opsional)
  { $sort: { rata_rata_nilai: -1 } }
])
```

Output:

```
[
  { _id: 'S10', rata_rata_nilai: 61.70092369477912 },
  { _id: 'S09', rata_rata_nilai: 60.53642276422764 },
  { _id: 'S06', rata_rata_nilai: 59.761212121212125 },
  { _id: 'S07', rata_rata_nilai: 59.37692307692308 },
  { _id: 'S02', rata_rata_nilai: 59.27139931740614 },
  { _id: 'S01', rata_rata_nilai: 58.930551470588235 },
  { _id: 'S04', rata_rata_nilai: 58.55 },
  { _id: 'S03', rata_rata_nilai: 58.24058394160584 },
  { _id: 'S05', rata_rata_nilai: 58.17157258064516 },
  { _id: 'S08', rata_rata_nilai: 57.9007299270073 }
]
```

Penjelasan:

\$match berfungsi sebagai filter awal (seperti WHERE di SQL) agar data yang diproses ke tahap selanjutnya hanya data dari 7 hari terakhir. Penempatan di awal sangat baik untuk optimasi. \$group mengelompokkan data berdasarkan sensor_id, lalu menggunakan akumulator. \$avg menghitung rata-rata dari field nilai dan \$sort mengurutkan hasil keluaran.

4.3 Sensor dengan nilai di atas 90 minimal 3 kali

Program:

```
db.data_sensor.aggregate([
  // Stage 1: Group dan hitung dengan $sum kondisional
  { $group: {
    _id: "$sensor_id",
    frekuensi_diatas_90: {
      $sum: {
        $cond: { if: { $gt: ["$nilai", 90] }, then: 1, else: 0 }
      }
    }
  }},
  // Stage 2: Filter yang jumlahnya minimal 3
  { $match: {
    frekuensi_diatas_90: { $gte: 3 }
  }},
  // Stage 3: Urutkan dari yang paling sering
  { $sort: { frekuensi_diatas_90: -1 } }
])
```

Output:

```
[
  { _id: 'S02', frekuensi_diatas_90: 75 },
  { _id: 'S07', frekuensi_diatas_90: 63 },
  { _id: 'S06', frekuensi_diatas_90: 56 },
  { _id: 'S10', frekuensi_diatas_90: 56 },
  { _id: 'S03', frekuensi_diatas_90: 54 },
  { _id: 'S05', frekuensi_diatas_90: 54 },
  { _id: 'S04', frekuensi_diatas_90: 53 },
  { _id: 'S01', frekuensi_diatas_90: 53 },
  { _id: 'S09', frekuensi_diatas_90: 52 },
  { _id: 'S08', frekuensi_diatas_90: 50 }
]
```

Penjelasan:

\$group mengelompokkan data per sensor lalu di dalamnya, saya menggunakan operator ekspresi logika \$cond di dalam penjumlah \$sum. Jika nilai > 90, maka dihitung 1. Jika tidak, dihitung 0. \$match berperan setelah ditotal per sensor sebagai penyaring hasil dan hanya menampilkan sensor yang memiliki frekuensi_diatas_90 lebih besar atau sama dengan (\$gte) 3.

4.4 Menambahkan field jam dan menghitung rata-rata per jam per sensor

Program:

```
db.data_sensor.aggregate([
  // Stage 1: Tambahkan field 'jam'
  { $addFields: {
    jam: { $hour: "$timestamp" }
  }},

  // Stage 2: Group berdasarkan sensor_id DAN jam
  { $group: {
    _id: {
      sensor: "$sensor_id",
      jam: "$jam"
    },
    rata_rata_per_jam: { $avg: "$nilai" }
  }},

  // Stage 3: Rapikan tampilan dengan $project
  { $project: {
    _id: 0,
    sensor_id: "$_id.sensor",
    jam: "$_id.jam",
    rata_rata: "$rata_rata_per_jam"
  }},

  // Stage 4: Urutkan berdasarkan sensor lalu jam
  { $sort: { sensor_id: 1, jam: 1 } }
])
```

Output:

```
[
  { sensor_id: 'S01', jam: 0, rata_rata: 69.34357142857142 },
  { sensor_id: 'S01', jam: 1, rata_rata: 65.626 },
  { sensor_id: 'S01', jam: 2, rata_rata: 63.565000000000005 },
  { sensor_id: 'S01', jam: 3, rata_rata: 62.731666666666666 },
  { sensor_id: 'S01', jam: 4, rata_rata: 52.897000000000006 },
  { sensor_id: 'S01', jam: 5, rata_rata: 65.320833333333334 },
  { sensor_id: 'S01', jam: 6, rata_rata: 50.757241379310344 },
  { sensor_id: 'S01', jam: 7, rata_rata: 58.896111111111104 },
  { sensor_id: 'S01', jam: 8, rata_rata: 57.626086956521746 },
  { sensor_id: 'S01', jam: 9, rata_rata: 59.33538461538462 },
  { sensor_id: 'S01', jam: 10, rata_rata: 59.753181818181815 },
  { sensor_id: 'S01', jam: 11, rata_rata: 58.419999999999995 },
  { sensor_id: 'S01', jam: 12, rata_rata: 61.000454545454545 },
  { sensor_id: 'S01', jam: 13, rata_rata: 60.970454545454544 },
  { sensor_id: 'S01', jam: 14, rata_rata: 60.68846153846154 },
  { sensor_id: 'S01', jam: 15, rata_rata: 55.62695652173913 },
  { sensor_id: 'S01', jam: 16, rata_rata: 55.47173913043478 },
  { sensor_id: 'S01', jam: 17, rata_rata: 51.13941176470588 },
  { sensor_id: 'S01', jam: 18, rata_rata: 54.39904761904762 },
  { sensor_id: 'S01', jam: 19, rata_rata: 62.092 }
]
```

Penjelasan:

\$addFields menambahkan field baru bernama jam. Saya menggunakan operator tanggal \$hour untuk mengambil angka jam (0-23) dari timestamp. \$group menggunakan composite ID (ID gabungan) yang terdiri dari sensor dan jam, agar rata-rata dihitung per jam untuk setiap sensornya. \$project membuang field _id dan menata ulang strukturnya agar lebih enak dibaca di hasil akhir.

4.5 Membuat koleksi baru lalu JOIN (\$lookup)

Program:

```
db.sensor_info.insertMany([
  { sensor_id: "S01", nama_sensor: "Suhu Mesin A", lokasi: "Gedung 1" },
  { sensor_id: "S02", nama_sensor: "Tekanan Pompa B", lokasi: "Gedung 1" },
  { sensor_id: "S03", nama_sensor: "Kelembaban Ruang C", lokasi: "Gedung 2" }
  // Tambahkan dummy lain sesuai kebutuhan
])
```

```
db.sensor_info.insertMany([
  { sensor_id: "S01", nama_sensor: "Suhu Mesin A", lokasi: "Gedung 1" },
  { sensor_id: "S02", nama_sensor: "Tekanan Pompa B", lokasi: "Gedung 1" },
  { sensor_id: "S03", nama_sensor: "Kelembaban Ruang C", lokasi: "Gedung 2" }
  // Tambahkan dummy lain sesuai kebutuhan
])
```

Output:

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69f85ac3b812d6ee367c2907'),
    '1': ObjectId('69f85ac3b812d6ee367c2908'),
    '2': ObjectId('69f85ac3b812d6ee367c2909')
  }
}
```

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69f85ac3b812d6ee367c2907'),
    '1': ObjectId('69f85ac3b812d6ee367c2908'),
    '2': ObjectId('69f85ac3b812d6ee367c2909')
  }
}
```

Analisis:

\$lookup melakukan operasi join. Ia mencari dokumen di sensor_info yang nilai sensor_id nya sama dengan nilai _id dari tahap sebelumnya, lalu meletakkannya di dalam array detail_sensor. Karena hasil \$lookup berbentuk array, saya pecah array tersebut menjadi dokumen tunggal dengan \$unwind agar hasil nya lebih mudah diakses di tahap selanjutnya.

BAB V

KESIMPULAN

Pada tahap Penugasan Terbimbing, fokus utama adalah membangun pemahaman fundamental mengenai pengoperasian MongoDB dan koneksinya dengan Python menggunakan pustaka PyMongo. Praktik ini membuktikan bahwa pembuatan ribuan dokumen dummy—seperti riwayat inspeksi pabrik dan target kualitas—dapat dieksekusi secara otomatis, cepat, dan stabil melalui scripting. Tahapan awal ini memberikan fondasi yang sangat penting mengenai cara mengonfigurasi database.

Memasuki tahap Tugas Mandiri, melalui penerapan Aggregation Framework saya mengevaluasi data inspeksi quality control. Melalui eksekusi lima pipeline, terbukti bahwa MongoDB mampu berfungsi sebagai mesin analitik. Penggunaan operator seperti \$lookup dapat membandingkan persentase cacat dengan target maksimal, pengelompokan kegagalan mingguan, hingga penggunaan \$facet untuk pemrosesan bulanan, berhasil mengidentifikasi mesin-mesin pabrik yang paling bermasalah sehingga siap ditindaklanjuti.

Pada tahap akhir yaitu Latihan Mandiri, fokus pembelajaran bergeser pada analisis data operasional (time-series) dan pemahaman desain struktur data melalui studi kasus sensor. Penggunaan agregasi bersyarat untuk mendeteksi sensor secara berulang, praktik ini menegaskan bahwa meskipun MongoDB bersifat fleksibel (tanpa skema kaku), sistem ini tetap mampu memberlakukan aturan validasi field dan tipe data yang baik.

Lampiran:

https://drive.google.com/drive/folders/1gaqwOmG554eXlfH7oiPI5cIEkceUUek?usp=s_haring